

# JAVA

|                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------|
| Java é uma linguagem de programação orientada a objetos, criada para ser portátil e segura. A arquitetura central é composta por: |
| <b>JVM (Java Virtual Machine):</b> executa bytecode e torna o programa portátil entre plataformas.                                |
| <b>JRE (Java Runtime Environment):</b> inclui a JVM e bibliotecas necessárias para executar programas Java.                       |
| <b>JDK (Java Development Kit):</b> inclui o compilador <code>javac</code> , ferramentas e bibliotecas para desenvolver em Java.   |
| Java segue o princípio "Write Once, Run Anywhere" – escreva uma vez, execute em qualquer plataforma com JVM.                      |

## 1. Declaração de variáveis

Para realizar a declaração de uma variável em Java deve-se utilizar a seguinte estrutura:

```
tipoDaVariavel nomeDaVariavel = valorDaVariavel;
```

Tipos primitivos:

- byte - 8 bits
- short - 16 bits
- int - 32 bits
- long - 64 bits
- double - 64 bits
- char - 16 bits
- boolean - true / false
- String (classe para texto)

```
int idade = 30;
long populacao = 8_000_000_000L;
double temperatura = 36.6;
char letra = 'A';
boolean ativo = true;
String Nome = "THalles";

"Exemplo de variável com nome composto (camelCase)":
String senhaAcesso = "Java123";
int valorTotal = 1000;
```

Convenção de nomes: use **camelCase** para variáveis e métodos.

## 2. Estruturas de Controle e Loops

As estruturas de controle definem o fluxo de execução de um programa.

### 2.1. Condicional if-else

```
if (x > 0) {
    System.out.println("Positivo");
} else {
    System.out.println("Negativo ou zero");
}
```

"Para codigos que necessitem de mais condições pode ser utilizar o else if"

```
if(x=0){
    System.out.println("Neutro");
} else if( x > 0){
    System.out.println("Positivo");
} else{
    System.out.println("Negativo");
}
```

```
}
```

## 2.2. Estrutura switch-case

Permite escolher entre várias opções com base em um valor específico.

```
int opcao = 2;
switch (opcao) {
    case 1:
        System.out.println("Opção 1 selecionada");
        break;
    case 2:
        System.out.println("Opção 2 selecionada");
        break;
    case 3:
        System.out.println("Opção 3 selecionada");
        break;
    default:
        System.out.println("Opção inválida");
}
```

O comando `break` encerra o caso atual; se omitido, o programa continuará executando os próximos casos (efeito conhecido como *fall-through*).

## 3. Estruturas de Repetição (Loops)

Os laços permitem repetir um bloco de código enquanto uma condição for verdadeira.

### 3.1. Estrutura for

Usada quando se conhece o número de repetições.

```
for (int i = 0; i < 5; i++) {
    System.out.println("Contagem: " + i);
}
```

### 3.2. Estrutura while

Executa o bloco **enquanto** a condição for verdadeira.

```
int contador = 0;
while (contador < 5) {
    System.out.println("Contador: " + contador);
    contador++;
}
```

Se a condição nunca se tornar falsa, o programa entra em um **loop infinito**.

### 3.3. Estrutura do-while

Executa o bloco **pelo menos uma vez**, mesmo que a condição seja falsa.

```
int numero = 0;
do {
    System.out.println("Número: " + numero);
    numero++;
} while (numero < 5);
```

Diferença:

- `while` testa a condição **antes** de executar.
- `do-while` executa **antes** de testar.

## 4. Arrays, Vetores e Matrizes

### 4.1. Vetores (Arrays Unidimensionais)

```
int[] numeros = {10, 20, 30, 40, 50};
for (int i = 0; i < numeros.length; i++) {
    System.out.println(numeros[i]);
}
```

### 4.2. Matriz (Arrays Bidimensionais)

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6}
};

for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

## 5. Resumo das Estruturas

| Estrutura | Descrição                           | Quando usar                                |
|-----------|-------------------------------------|--------------------------------------------|
| if / else | Testa condições simples             | Quando há decisões binárias                |
| switch    | Escolhe entre várias opções         | Quando há múltiplos caminhos fixos         |
| for       | Laço com contagem controlada        | Quando sabe o número de repetições         |
| while     | Laço condicional                    | Quando a repetição depende de uma condição |
| do-while  | Laço que executa pelo menos uma vez | Quando precisa testar a condição depois    |

## 6. Exercícios de Prática

- Crie um programa que exiba números de 1 a 10 usando `for`.
- Faça um contador regressivo de 10 a 0 com `while`.
- Use `do-while` para pedir ao usuário uma senha até que ele acerte.
- Crie um menu interativo com `switch-case` que permita escolher opções de um sistema simples.

Vamos entrar no **tópico: Funções (métodos) e escopo de variáveis em Java**, parte essencial antes de Programação Orientada a Objetos (POO).

### 1. O que é uma função (método) em Java

Em Java, **funções são chamadas de métodos**. Um **método** é um bloco de código que executa uma tarefa específica e pode ser reutilizado em diferentes partes do programa.

👉 **Vantagens de usar métodos:**

- Evita repetição de código.
- Facilita manutenção.
- Torna o código mais organizado e legível.

## ⚙️ 2. Estrutura básica de um método

```
modificador tipoDeRetorno nomeDoMetodo(parâmetros) {  
    // corpo do método  
    return valor; // opcional, depende do tipoDeRetorno  
}
```

### Exemplo:

```
public static int somar(int a, int b) {  
    int resultado = a + b;  
    return resultado;  
}
```

#### 🧠 Explicando:

- public → modificador de acesso (visível a todos).
- static → pode ser chamado sem criar objeto.
- int → tipo de retorno.
- somar → nome do método.
- (int a, int b) → parâmetros.
- return resultado; → devolve o valor ao chamador.

## 🔄 3. Chamando um método

```
public class ExemploMetodo {  
    public static void main(String[] args) {  
        int soma = somar(10, 5);  
        System.out.println("A soma é: " + soma);  
    }  
  
    public static int somar(int a, int b) {  
        return a + b;  
    }  
}
```

#### ✂️ Saída:

```
A soma é: 15
```

## 💬 4. Métodos sem retorno (void)

Nem todos os métodos precisam retornar algo.

```
public static void saudacao(String nome) {  
    System.out.println("Olá, " + nome + "!");  
}
```

#### Chamando:

```
saudacao("Thalles");
```

#### Saída:

## 📦 5. Escopo de variáveis

O **escopo** define onde uma variável pode ser acessada.

| Tipo                                       | Onde é declarada                            | Onde é visível                     |
|--------------------------------------------|---------------------------------------------|------------------------------------|
| Variável local                             | Dentro de um método                         | Apenas dentro desse método         |
| Variável de classe ( <code>static</code> ) | Dentro da classe, fora de métodos           | Pode ser acessada sem criar objeto |
| Variável de instância                      | Dentro da classe (sem <code>static</code> ) | Pertence a cada objeto criado      |

### Exemplo:

```
public class EscopoExemplo {
    static int contadorGeral = 0; // variável de classe
    int contadorObjeto = 0;      // variável de instância

    public void incrementar() {
        contadorGeral++;
        contadorObjeto++;
        System.out.println("Geral: " + contadorGeral + " | Objeto: " + contadorObjeto);
    }

    public static void main(String[] args) {
        EscopoExemplo e1 = new EscopoExemplo();
        EscopoExemplo e2 = new EscopoExemplo();

        e1.incrementar();
        e2.incrementar();
        e1.incrementar();
    }
}
```

✂ Saída:

```
Geral: 1 | Objeto: 1
Geral: 2 | Objeto: 1
Geral: 3 | Objeto: 2
```

👉 Note que `contadorGeral` é **compartilhado entre os objetos**, enquanto `contadorObjeto` é **individual de cada instância**.

## 🎲 6. Tipos de métodos

| Tipo                         | Descrição                       | Exemplo                                   |
|------------------------------|---------------------------------|-------------------------------------------|
| Com retorno e com parâmetros | Retorna um valor e recebe dados | <code>int somar(int a, int b)</code>      |
| Com retorno e sem parâmetros | Retorna algo fixo               | <code>double pi()</code>                  |
| Sem retorno e com parâmetros | Apenas executa algo             | <code>void exibirNome(String nome)</code> |
| Sem retorno e sem parâmetros | Ação simples                    | <code>void mensagem()</code>              |

## 🧠 7. Boas práticas

- Use nomes descritivos para métodos: `calcularMedia()`, `mostrarMenu()`, `validarSenha()`.
- Evite métodos muito longos.
- Um método deve **fazer apenas uma coisa**.
- Evite variáveis globais sem necessidade.

## 8. Mini desafio

Crie um programa com:

- Um método chamado `media` que recebe 3 notas e retorna a média.
- Um método `situacao` que recebe a média e imprime se o aluno está **Aprovado** ( $\geq 7$ ), **Recuperação** ( $\geq 5$ ) ou **Reprovado** ( $< 5$ ).

Perfeito, Jarvis continua a missão 🚀

Chegamos agora ao **coração da linguagem Java: a Programação Orientada a Objetos (POO)** — o paradigma que transforma o código em algo mais **organizado, modular e parecido com o mundo real**.

## 🧭 Programação Orientada a Objetos (POO) em Java

### 🧩 1. O que é POO

A **Programação Orientada a Objetos (POO)** é um modelo de programação que organiza o código em **classes** e **objetos**.

- 👉 **Classe** = modelo, molde ou “planta baixa”.
- 👉 **Objeto** = instância (ou cópia viva) dessa classe.

📖 **Exemplo do mundo real:**

- Classe:** “Carro”
- Objetos:** “Meu carro”, “Carro da Maria”, “Carro do João”

Cada carro tem **atributos** (cor, modelo, ano) e **comportamentos** (acelerar, frear, ligar).

### ⚙️ 2. Estrutura básica de uma classe em Java

```
public class Carro {
    // Atributos
    String modelo;
    String cor;
    int ano;

    // Métodos
    void ligar() {
        System.out.println("O carro está ligado.");
    }

    void exibirInfo() {
        System.out.println("Modelo: " + modelo);
        System.out.println("Cor: " + cor);
        System.out.println("Ano: " + ano);
    }
}
```

**Criando um objeto:**

```
public class Main {
    public static void main(String[] args) {
        Carro carro1 = new Carro(); // cria um objeto
        carro1.modelo = "Fusca";
        carro1.cor = "Azul";
        carro1.ano = 1975;

        carro1.ligar();
        carro1.exibirInfo();
    }
}
```

🧩 **Saída:**



0 carro está ligado.  
Modelo: Fusca  
Cor: Azul  
Ano: 1975

### 3. Pilares da POO

A POO é baseada em 4 pilares principais:

| Pilar          | Descrição                                                                                     | Exemplo                                                       |
|----------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Encapsulamento | Protege os dados, controlando acesso através de métodos <code>get</code> e <code>set</code> . | <code>private</code> atributos + <code>getters/setters</code> |
| Herança        | Permite que uma classe herde atributos e métodos de outra.                                    | <code>class Filho extends Pai</code>                          |
| Polimorfismo   | Permite métodos com o mesmo nome, mas comportamentos diferentes.                              | Sobrescrita e sobrecarga                                      |
| Abstração      | Mostra apenas o essencial e oculta detalhes.                                                  | Classes e métodos abstratos                                   |

### 4. Encapsulamento

Encapsular é **proteger os dados internos da classe**, tornando-os privados e acessíveis apenas por métodos controlados.

```
public class ContaBancaria {
    private double saldo; // privado

    public void depositar(double valor) {
        saldo += valor;
    }

    public void sacar(double valor) {
        if (valor <= saldo) {
            saldo -= valor;
        } else {
            System.out.println("Saldo insuficiente!");
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

#### Uso:

```
public class Main {
    public static void main(String[] args) {
        ContaBancaria conta = new ContaBancaria();
        conta.depositar(500);
        conta.sacar(100);
        System.out.println("Saldo atual: " + conta.getSaldo());
    }
}
```

### 5. Herança

Permite **reaproveitar código** entre classes relacionadas.

```
// Classe base
public class Animal {
    void comer() {
        System.out.println("0 animal está comendo...");
    }
}
```

```
    }  
}  
  
// Classe filha  
public class Cachorro extends Animal {  
    void latir() {  
        System.out.println("O cachorro está latindo!");  
    }  
}
```

**Uso:**

```
public class Main {  
    public static void main(String[] args) {  
        Cachorro dog = new Cachorro();  
        dog.comer(); // herdado de Animal  
        dog.latir(); // método da classe Cachorro  
    }  
}
```

 **6. Polimorfismo**

Um mesmo método pode ter **diferentes comportamentos** dependendo do contexto.

**Exemplo de sobrescrita:**

```
public class Animal {  
    void som() {  
        System.out.println("Som genérico de animal");  
    }  
}  
  
public class Gato extends Animal {  
    @Override  
    void som() {  
        System.out.println("Miau!");  
    }  
}  
  
public class Cachorro extends Animal {  
    @Override  
    void som() {  
        System.out.println("Au au!");  
    }  
}
```

**Uso:**

```
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Gato();  
        Animal a2 = new Cachorro();  
  
        a1.som(); // Miau!  
        a2.som(); // Au au!  
    }  
}
```

 **7. Abstração**

Mostra **somente o essencial** e **oculta detalhes**.

```
public abstract class Funcionario {  
    String nome;  
    double salario;
```



```
        abstract void calcularBonus();
    }

    public class Gerente extends Funcionario {
        @Override
        void calcularBonus() {
            System.out.println("Bônus do gerente: " + (salario * 0.2));
        }
    }
}
```

## 8. Construtores

Um **construtor** é chamado automaticamente quando o objeto é criado.

```
public class Pessoa {
    String nome;
    int idade;

    // Construtor
    public Pessoa(String n, int i) {
        nome = n;
        idade = i;
    }

    void exibir() {
        System.out.println("Nome: " + nome + " | Idade: " + idade);
    }
}
```

### Uso:

```
public class Main {
    public static void main(String[] args) {
        Pessoa p1 = new Pessoa("Thalles", 25);
        p1.exibir();
    }
}
```

## 9. Mini Desafio POO

Crie um sistema simples de cadastro de **alunos**:

1. Classe `Aluno` com atributos `nome`, `nota1`, `nota2`.
2. Método para calcular média.
3. Método que retorna a situação (Aprovado/Reprovado).
4. No `main`, crie 2 alunos e mostre os resultados.

Vamos então dar continuidade com um dos temas mais importantes da **POO avançada**: **Interfaces, Classes Abstratas e Composição em Java**.  
Esses conceitos tornam o código mais **flexível, reutilizável e organizado**, permitindo criar **sistemas mais profissionais e fáceis de manter**.

## Interfaces, Classes Abstratas e Composição em Java

### 1. Classe Abstrata — “Modelo base com partes indefinidas”

Uma **classe abstrata** serve como **modelo genérico** para outras classes. Ela **não pode ser instanciada diretamente**, ou seja, você **não pode criar um objeto** dela.

Usamos o modificador `abstract` para indicá-la.

 **Exemplo:**

```
public abstract class Funcionario {
    String nome;
    double salario;

    // Método comum
    void exibirDados() {
        System.out.println("Nome: " + nome);
        System.out.println("Salário: " + salario);
    }

    // Método abstrato (sem corpo)
    abstract void calcularBonus();
}
```

**Subclasse que herda e implementa o método:**

```
public class Gerente extends Funcionario {
    @Override
    void calcularBonus() {
        System.out.println("Bônus do gerente: " + (salario * 0.20));
    }
}

public class Vendedor extends Funcionario {
    @Override
    void calcularBonus() {
        System.out.println("Bônus do vendedor: " + (salario * 0.10));
    }
}
```

**Uso:**

```
public class Main {
    public static void main(String[] args) {
        Gerente g = new Gerente();
        g.nome = "Carlos";
        g.salario = 5000;
        g.exibirDados();
        g.calcularBonus();

        Vendedor v = new Vendedor();
        v.nome = "Ana";
        v.salario = 3000;
        v.exibirDados();
        v.calcularBonus();
    }
}
```

 **Saída:**

```
Nome: Carlos
Salário: 5000.0
Bônus do gerente: 1000.0
Nome: Ana
Salário: 3000.0
Bônus do vendedor: 300.0
```

 **Resumindo:**

- Classe abstrata **pode ter métodos concretos e abstratos**.
- Serve como **modelo parcial** para outras classes.
- **Não pode ser instanciada**.

## 🧩 2. Interface — “Contrato que deve ser seguido”

Uma **interface** define **regras que as classes devem cumprir**, ou seja, **métodos obrigatórios**. Ela é parecida com uma classe abstrata, mas **não tem implementação de métodos**.

Usamos o modificador `interface` .

### ✅ Exemplo:

```
public interface Pagavel {
    void calcularPagamento();
}
```

### Implementando a interface:

```
public class Freelancer implements Pagavel {
    double horasTrabalhadas;
    double valorHora;

    @Override
    public void calcularPagamento() {
        double pagamento = horasTrabalhadas * valorHora;
        System.out.println("Pagamento do freelancer: R$ " + pagamento);
    }
}
```

### Uso:

```
public class Main {
    public static void main(String[] args) {
        Freelancer f = new Freelancer();
        f.horasTrabalhadas = 40;
        f.valorHora = 50;
        f.calcularPagamento();
    }
}
```

### 🖨 Saída:

```
Pagamento do freelancer: R$ 2000.0
```

### 👉 Resumindo:

| Item                  | Classe Abstrata                             | Interface                                                                                  |
|-----------------------|---------------------------------------------|--------------------------------------------------------------------------------------------|
| Implementa métodos?   | Sim, pode ter métodos concretos e abstratos | Não (até o Java 7); a partir do Java 8 pode ter <code>default</code> e <code>static</code> |
| Pode ter atributos?   | Sim                                         | Somente constantes ( <code>final static</code> )                                           |
| Pode herdar de outra? | Sim (1 classe)                              | Pode implementar várias interfaces                                                         |
| Instanciável?         | Não                                         | Não                                                                                        |
| Palavra-chave         | <code>abstract</code>                       | <code>interface</code> , <code>implements</code>                                           |

## 🧩 3. Herança Múltipla via Interfaces

Java **não permite herança múltipla entre classes**, mas **permite múltiplas interfaces**:

```
public interface Animal {
    void comer();
}

public interface Pet {
    void brincar();
}
```

```
}

public class Cachorro implements Animal, Pet {
    @Override
    public void comer() {
        System.out.println("O cachorro está comendo.");
    }

    @Override
    public void brincar() {
        System.out.println("O cachorro está brincando.");
    }
}
```

## 4. Composição — “Tem um”

Enquanto a **herança** expressa “é um”, a **composição** expressa “tem um”.

👉 Exemplo:

Um **Carro** *tem um* **Motor**.

Mas o carro **não é** um motor.

✅ Exemplo:

```
public class Motor {
    int potencia;

    void ligar() {
        System.out.println("Motor ligado com potência de " + potencia + "cv");
    }
}

public class Carro {
    String modelo;
    Motor motor; // composição

    void ligarCarro() {
        if (motor != null) {
            motor.ligar();
            System.out.println("Carro " + modelo + " ligado!");
        } else {
            System.out.println("Sem motor!");
        }
    }
}
```

Uso:

```
public class Main {
    public static void main(String[] args) {
        Motor m = new Motor();
        m.potencia = 150;

        Carro c = new Carro();
        c.modelo = "Sedan";
        c.motor = m; // associação de objetos

        c.ligarCarro();
    }
}
```

🖨 Saída:

```
Motor ligado com potência de 150cv
Carro Sedan ligado!
```

## 🧩 5. Quando usar cada um?

| Situação                                                             | Melhor opção    |
|----------------------------------------------------------------------|-----------------|
| Você quer definir um modelo base com partes comuns e personalizáveis | Classe abstrata |
| Você quer definir regras que várias classes diferentes devem seguir  | Interface       |
| Você quer montar objetos a partir de outros objetos                  | Composição      |

### 💡 Mini Desafio

Crie um sistema com:

- Interface `OperacaoMatematica` (método `calcular(double a, double b)`);
- Classes `Soma`, `Subtracao`, `Multiplicacao` e `Divisao` que implementam essa interface;
- No `main`, leia dois números e execute todas as operações usando as classes criadas.

Perfeito, Jarvis! ⚙️🔥

Agora chegamos à parte **visual e prática** da linguagem: **Interface Gráfica (GUI)** em Java. Vamos construir passo a passo — do conceito até o código funcional com **Swing**, **JFrame**, **JPanel**, **JButton**, **JLabel** e **JTextField**.

## 🎨 Interface Gráfica (GUI) em Java — Swing

### 🧠 1. O que é Swing?

O **Swing** é um conjunto de classes do Java (no pacote `javax.swing`) que permite criar **interfaces gráficas** — janelas, botões, menus, caixas de texto etc.

Ele é parte do **Java Foundation Classes (JFC)** e é independente de plataforma (funciona em qualquer sistema com JVM).

✅ **Principais classes:**

| Classe                   | Função                                               |
|--------------------------|------------------------------------------------------|
| <code>JFrame</code>      | Janela principal                                     |
| <code>JPanel</code>      | Painel de organização dentro da janela               |
| <code>JLabel</code>      | Texto fixo (rótulo)                                  |
| <code>JButton</code>     | Botão clicável                                       |
| <code>JTextField</code>  | Campo de entrada de texto                            |
| <code>JOptionPane</code> | Caixas de diálogo (alertas, mensagens, confirmações) |

### ⚙️ 2. Criando a primeira janela com Swing

```
import javax.swing.JFrame;
import javax.swing.JLabel;

public class JanelaSimples {
    public static void main(String[] args) {
        JFrame janela = new JFrame("Minha Primeira Janela");
        JLabel texto = new JLabel("Olá, Mundo Gráfico!", JLabel.CENTER);

        janela.add(texto); // adiciona o rótulo à janela
        janela.setSize(300, 200); // largura x altura
        janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // encerra o programa ao fechar
        janela.setVisible(true); // torna visível
    }
}
```

```
}  
}
```

#### 📖 Explicação:

- `JFrame` → cria a janela.
- `JLabel` → adiciona um texto.
- `setDefaultCloseOperation` → define o comportamento ao fechar.
- `setVisible(true)` → exibe a janela.

## 🗦️ 3. Adicionando botão e campo de texto

```
import javax.swing.*;  
import java.awt.event.*; // para eventos  
  
public class InterfaceComBotao {  
    public static void main(String[] args) {  
        JFrame janela = new JFrame("Exemplo com Botão");  
        janela.setSize(350, 200);  
        janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        janela.setLayout(null); // layout livre (posição manual)  
  
        JLabel rotulo = new JLabel("Digite seu nome:");  
        rotulo.setBounds(20, 20, 120, 25);  
        janela.add(rotulo);  
  
        JTextField campo = new JTextField();  
        campo.setBounds(140, 20, 150, 25);  
        janela.add(campo);  
  
        JButton botao = new JButton("Saudar");  
        botao.setBounds(100, 70, 120, 30);  
        janela.add(botao);  
  
        JLabel resultado = new JLabel("");  
        resultado.setBounds(20, 110, 300, 25);  
        janela.add(resultado);  
  
        // Evento do botão  
        botao.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                String nome = campo.getText();  
                resultado.setText("Olá, " + nome + "!");  
            }  
        });  
  
        janela.setVisible(true);  
    }  
}
```

#### 📖 O que acontece aqui:

- Criamos um **campo de texto** (`JTextField`) e um **botão** (`JButton`).
- Quando o botão é clicado, um **evento** (`ActionEvent`) é disparado.
- O método `actionPerformed` pega o texto digitado e exibe a saudação.

## 💬 4. Exemplo com `JOptionPane` (caixas de diálogo)

O `JOptionPane` é uma maneira rápida de criar **alertas e entradas de usuário**.

```
import javax.swing.JOptionPane;  
  
public class CaixaDialogo {  
    public static void main(String[] args) {  
        String nome = JOptionPane.showInputDialog("Digite seu nome:");  
        JOptionPane.showMessageDialog(null, "Olá, " + nome + "!");  
    }  
}
```



```
    }  
}
```

 **Funções principais do JOptionPane:**

| Método            | Descrição                       |
|-------------------|---------------------------------|
| showMessageDialog | Exibe uma mensagem simples      |
| showInputDialog   | Solicita uma entrada de texto   |
| showConfirmDialog | Exibe opções (Sim/Não/Cancelar) |

## 5. Usando Layout Managers

Layouts controlam automaticamente o **posicionamento dos componentes**, evitando usar coordenadas manuais.

### Exemplos:

| Layout       | Descrição                                          |
|--------------|----------------------------------------------------|
| FlowLayout   | Posiciona os elementos em linha                    |
| BorderLayout | Divide a tela em Norte, Sul, Leste, Oeste e Centro |
| GridLayout   | Organiza em formato de tabela (linhas e colunas)   |

### Exemplo com GridLayout:

```
import javax.swing.*.*;  
import java.awt.*.*;  
  
public class ExemploGridLayout {  
    public static void main(String[] args) {  
        JFrame janela = new JFrame("Calculadora");  
        janela.setLayout(new GridLayout(3, 2)); // 3 linhas, 2 colunas  
        janela.setSize(300, 150);  
        janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
        janela.add(new JLabel("Número 1:"));  
        JTextField n1 = new JTextField();  
        janela.add(n1);  
  
        janela.add(new JLabel("Número 2:"));  
        JTextField n2 = new JTextField();  
        janela.add(n2);  
  
        JButton somar = new JButton("Somar");  
        JLabel resultado = new JLabel("Resultado: ");  
        janela.add(somar);  
        janela.add(resultado);  
  
        somar.addActionListener(e -> {  
            try {  
                double a = Double.parseDouble(n1.getText());  
                double b = Double.parseDouble(n2.getText());  
                resultado.setText("Resultado: " + (a + b));  
            } catch (NumberFormatException ex) {  
                JOptionPane.showMessageDialog(janela, "Digite apenas números!");  
            }  
        });  
  
        janela.setVisible(true);  
    }  
}
```

## 6. Integrando GUI com Lógica e POO

Você pode integrar **POO + GUI**, separando lógica e interface:

```
class Calculadora {
    double somar(double a, double b) { return a + b; }
}

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class CalculadoraGUI {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();

        JFrame janela = new JFrame("Mini Calculadora");
        janela.setLayout(new FlowLayout());
        janela.setSize(300, 150);
        janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTextField n1 = new JTextField(5);
        JTextField n2 = new JTextField(5);
        JButton btn = new JButton("Somar");
        JLabel res = new JLabel("Resultado: ");

        btn.addActionListener(e -> {
            try {
                double a = Double.parseDouble(n1.getText());
                double b = Double.parseDouble(n2.getText());
                res.setText("Resultado: " + calc.somar(a, b));
            } catch (NumberFormatException ex) {
                res.setText("Erro: número inválido");
            }
        });

        janela.add(new JLabel("A:")); janela.add(n1);
        janela.add(new JLabel("B:")); janela.add(n2);
        janela.add(btn);
        janela.add(res);

        janela.setVisible(true);
    }
}
```

## Resumo geral de Swing

| Elemento       | Função                       |
|----------------|------------------------------|
| JFrame         | Janela principal             |
| JPanel         | Painel interno               |
| JButton        | Botão clicável               |
| JLabel         | Texto                        |
| JTextField     | Campo de texto               |
| JOptionPane    | Caixas de diálogo            |
| ActionListener | Captura de cliques e eventos |

## Próximos passos (se quiser seguir):

1. Criar menus ( JMenuBar , JMenu , JMenuItem )
2. Usar **imagens e ícones** em botões ( ImageIcon )
3. Criar **interface com múltiplas telas**
4. Migrar de Swing para **JavaFX** (mais moderno)



